LASR-MCP: Learned Adaptive Schema Resolution with Dynamic In-Tree Recovery for Enterprise Tool-Calling Agents

Anand S. Suchak
Independent Researcher
Email: anand02suchak@gmail.com

***Abstract*—Enterprise Model Context Protocol (MCP) agents frequently expose large, deeply constrained JSON Schema specifications that inflate prompt token budgets. While lazy schema resolution seeks to mitigate context bloat, Learned Adaptive Schema Resolution (LASR-MCP) operates by pruning upfront schemas and dynamically injecting violated constraints upon validation failure. However, preliminary v1.0 evaluations suffered from heuristic token double-counting and neglected the multi-turn cost of retry cascades. In this paper, we present a rigorous empirical and theoretical evaluation of LASR-MCP (`mcp-inventory-bench` v2.0.3) across $N = 3,840$ completed trials across 32 tasks, six experimental arms, and 20 fixed-seed executions per task-arm pair ($7,860$ response-bearing API invocations on `gemini-3.5-flash-lite` at temperature 0.2, alongside 154 client transport retries). Using provider-reported prompt-plus-candidate ($P + C$) token usage from `usageMetadata` and a paired task-cluster percentile bootstrap (10,000 resamples, seed 42), we establish three foundational findings. First, naive zero-constraint lean schemas (`ci_k0`) induce severe multi-attempt retry cascades (0.0% first-pass success), degrading final structural reliability to 87.5% and inflating net token consumption by 84.3% ($2,057.7$ vs. $1,116.8$ mean recorded $P + C$ tokens for Full Control; 95% signed-savings CI: $[-125.15\%, -49.21\%]$). Second, prioritized upfront bundling (`ci_k2_reactive`) slashes net token consumption by 31.7% compared to `ci_k0` ($1,406.5$ mean recorded $P + C$ tokens), recovers 96.9% final structural reliability (93.8% semantic reliability), and operates within 2.1% of a task-aware oracle reference baseline (`oracle_reactive`, $1,377.8$ tokens, where LASR $k = 2$ is 2.08% higher using the oracle as denominator). Third, we characterize an observed *Retry-Amortization Boundary* across the four evaluated enterprise task archetypes: across the full benchmark, Full Control achieves a mean recorded $P + C$ subtotal of $1,116.8$ tokens per trial versus $1,406.5$ for LASR $k = 2$, meaning LASR $k = 2$'s overall workload is approximately 25.94% higher than Full Control's (95% CI on workload increase: $[+3.16\%, +52.55\%]$, corresponding to signed relative savings of -25.94%, 95% CI: $[-52.55\%, -3.16\%]$, where the 95% bootstrap CI excludes zero); we do not describe the overall benchmark result as token savings. Rather, token savings are strictly cluster-specific: in routine operational regimes (C_1) where active task constraints fall within the upfront budget ($m_{\text{task}}(x) \leq k$), LASR $k = 2$ achieves 75.0% first-pass success and delivers approximately 13.5% lower $P + C$ usage (847.6 vs. 979.8 tokens) at 100.0% reliability; conversely, in deep inter-field constraint regimes (C_2–C_4, $m_{\text{task}}(x) > k$), single-tool isolation favors monolithic schemas by avoiding multi-turn prompt retransmission. We distinguish the full multi-session collection timeline from the final resumed execution session and acknowledge that all 20 repetitions per task-arm produced identical outputs under the repeated fixed seed. All raw telemetry, audit scripts, and cryptographic checksums are publicly released to establish an auditable benchmark for agentic tool use.*

***Index Terms*—Model Context Protocol (MCP), Autonomous Agents, Adaptive Schema Resolution, JSON Schema, Enterprise Tool-Calling, In-Tree Recovery, Token Efficiency.**

I. Introduction

The deployment of Large Language Models (LLMs) in mission-critical enterprise workflows has shifted from conversational assistants toward autonomous, closed-loop execution agents [1], [2]. In enterprise operations—such as multi-warehouse replenishment, inventory reconciliation, and procurement logistics—agents interact with transactional backends via standardized tool-calling protocols. The Model Context Protocol (MCP) [3] (formalized in specification 2026-07-28) formalizes this client-server interaction, providing structured JSON-RPC interfaces for tool discovery and parameter submission. Our experiments target MCP tool-schema exposure and tool-call argument generation; protocol transport, session, and discovery semantics are outside the evaluation scope.

A. The Dilemma of Enterprise Tool Schemas

Despite MCP's architectural standardization, exposing enterprise-grade tools to LLMs presents a challenging systems dilemma:

1) **The Monolithic Full-Schema Tax**: Enterprise tools enforce intricate data contracts governed by JSON Schema Draft 2020-12 [4] (e.g., supplier regex patterns, item-level quantity bounds, warehouse routing enums, and budget restrictions). Transmitting complete monolithic schemas (often exceeding 500 tokens per tool) on every turn inflates prompt token overhead, drives up financial API costs, and degrades inference throughput [5].
2) **The Brittleness of Naive Truncation**: Conversely, statically stripping schemas down to basic primitive types eliminates vital domain rules. In our benchmarks, when task-conditioned rules (such as warehouse routing enums) are omitted, models hallucinate non-existent identifiers, resulting in catastrophic validation failures that may remain unresolved through unassisted retries.

B. Our Approach: Three-Tier Adaptive Resolution

To overcome this trade-off, we propose **Learned Adaptive Schema Resolution (LASR-MCP)**. Rather than treating

schema resolution as static compression or isolated error string dumping, LASR-MCP introduces a tripartite schema hierarchy:

$$C_{\text{active}}^{(t)} = C_I^* \cup \hat{C}_T(x) \cup C_R^{(t)} \tag{1}$$

combining:

- **Tier 1 (Invariant Foundation Floor, C_I^*)**: A core subset of constraints (339 tokens for the evaluated purchase-order schema) that establishes the non-negotiable structural foundation required for syntactically valid submissions.
- **Tier 2 (Proactive Intent Prediction, $\hat{C}_T(x)$)**: A lightweight semantic classifier that predicts task-relevant constraint bundles from user intent prior to execution.
- **Tier 3 (Dynamic In-Tree Recomposition, $C_R^{(t)}$)**: A reactive operator that extracts the full set of concurrent schema violations from validator feedback and splices the missing constraint nodes directly into the native JSON Schema hierarchy on Attempt 2.

C. Hypotheses & Empirical Findings

We investigate four central hypotheses across a synthetic enterprise-style supply-chain tool benchmark:

- H_1 **(Semantic Predictability)**: *Task intent contains sufficient signal to predict constraint requirements beyond frequency priors on held-out template families with recurring constraint bundles.* → **Supported for Template Family C**: LASR achieves $F_1 = 0.863$ versus 0.250 for the frequency prior ($\Delta F_1 = +0.613$, Monte Carlo sign-flip $p < 0.001$; four-cluster sensitivity $p = 0.125$).
- H_2 **(Invariant Safety Floor)**: *A minimal invariant foundation is necessary to prevent structural collapse.* → **Supported within the Evaluated Benchmark**: $C_I^* = 339$ tokens is the empirically selected invariant foundation meeting the pre-specified 70% structural-validity floor; further pruning drops structural validity from 75% to 50%.
- H_3 **(Reactive In-Tree Recovery)**: *Pass-level dynamic in-tree recovery improves structural completion over static adaptive exposure on the evaluated benchmark.* → **Supported on Primary Benchmark**: Structural success increases from 78.1% to 100.0% (ΔSuccess $= +21.9$ pp, 95% CI: $[+9.4, +37.5]$ pp, exact one-sided McNemar/binomial test $p = 0.0078125$, $p < 0.01$; cluster-level sensitivity $p = 0.125$), recovering all 27 trajectories that initially failed structural validation in the historical $N = 32$ development benchmark (where 5/32 initially passed structural validation and 27/32 failed, achieving 27/27 = 100.0% structural recovery by Attempt 3). In the canonical live benchmark ($N = 3,840$ trials), reactive recovery similarly improves final structural completion from 90.625% to 96.875% ($+6.25$ pp) and semantic correctness from 87.500% to 93.750% ($+6.25$ pp).
- H_4 **(Workload Token Efficiency)**: *Adaptive resolution reduces net recorded token consumption across enterprise workloads.* → **Regime-Dependent**: In initial single-trial exploratory evaluations on the primary frozen benchmark ($N = 32$), reactive LASR cut recorded prompt-plus-candidate token consumption by 16.6% ($k = 1$) and 18.7% ($k = 2$) compared to monolithic control ($61,853 \rightarrow 50,276$ tokens). However, in our comprehensive live benchmark ($N = 3,840$ completed trials across 32 tasks, 6 arms, and 20 fixed-seed executions per task-arm pair at temperature 0.2), Full Control achieved a mean recorded $P + C$ subtotal of $1,116.8$ tokens per trial versus $1,406.5$ for LASR $k = 2$, such that LASR $k = 2$'s overall workload is approximately 25.94% higher than Full Control's (95% CI on workload increase: $[+3.16\%, +52.55\%]$; signed relative savings: -25.94%, 95% CI: $[-52.55\%, -3.16\%]$, where the 95% bootstrap CI excludes zero); we do not describe the overall benchmark result as token savings. Relative to the unassisted zero-constraint baseline (`ci_k0`, $2,057.7$ mean tokens), LASR $k = 2$ achieves a descriptive mean workload reduction of 31.65% (paired task-cluster percentile bootstrap 95% CI: $[26.57\%, 36.72\%]$). Token savings vs. Full Control are strictly cluster-specific: in routine operational regimes ($C_1, m_{\text{task}}(x) \leq k$), LASR $k = 2$ delivers approximately 13.5% lower $P + C$ usage (847.6 vs. 979.8 tokens) at 100.0% reliability, whereas in deep inter-field constraint regimes (C_2–C_4, $m_{\text{task}}(x) > k$), Full Control avoids multi-turn prompt re-transmission.

D. Contributions

1) **Formalization of Three-Tier Schema Resolution**: We formalize the interaction between invariant structural baselines, proactive intent prediction, and pass-level in-tree recomposition.
2) **Controlled Systems Benchmark**: We execute a six-arm controlled systems evaluation across 32 tasks spanning four operational archetypes on the primary frozen benchmark, providing empirical evidence for the structural contribution of reactive recovery under paired conditions.
3) **The Retry-Amortization Boundary**: Through our comprehensive $N = 3,840$ completed trial evaluation (32 tasks, 6 arms, 20 fixed-seed executions per task-arm pair at temperature 0.2, comprising 7,860 response-bearing API invocations alongside 154 client transport retries), we characterize the break-even boundary between up-front schema compression and multi-turn retry cascades: demonstrating that LASR $k = 2$ achieves approximately 13.5% lower $P + C$ token usage in routine domains ($C_1, m_{\text{task}}(x) \leq k$), while overall across all 32 tasks, LASR $k = 2$'s workload is approximately 25.94% higher than Full Control's ($1,406.5$ vs. $1,116.8$ mean recorded $P + C$ tokens) in single-tool isolation because multi-turn prompt re-transmission penalties outweigh initial schema compression.
4) **Open Reproducibility Package**: We release the benchmark code, traces, and frozen dataset with an automated SHA-256 cryptographic integrity manifest.

II. The LASR-MCP Architecture

To resolve the trade-off between bloated context overhead and brittle schema truncation, we introduce **Learned Adaptive**

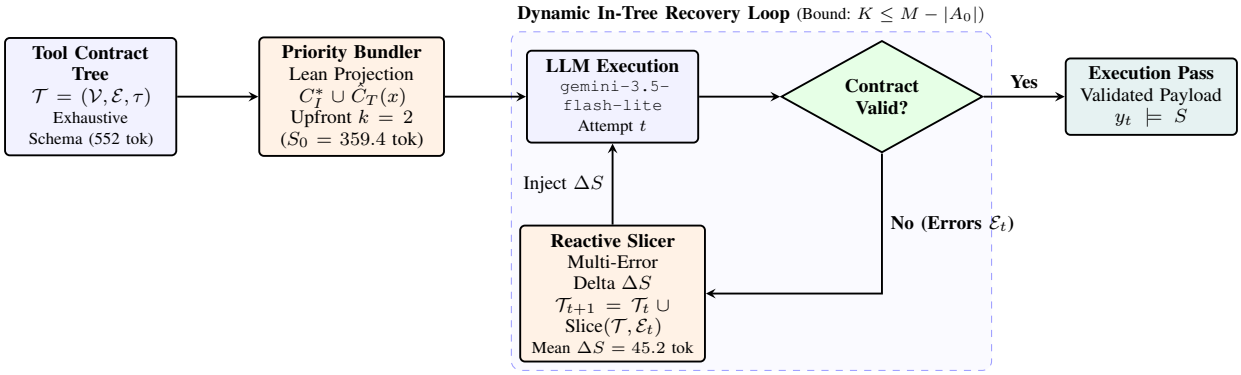


Fig. 1. System Architecture of LASR-MCP: The full tool contract tree $\mathcal{T}$ is structurally projected into an invariant foundation C_I^* with $k = 2$ prioritized constraint bundles ($S_0 = 359.4$ tokens). Upon validation failure, the reactive slicer dynamically splices concurrent violated constraint subtrees ($\Delta S = 45.2$ tokens) corresponding to the diagnostic error set $\mathcal{E}_t$ into Attempt $t + 1$, bounded by the monotonic convergence invariant $K \leq M - |A_0|$.

Schema Resolution (LASR-MCP). Rather than treating tool schemas as static monolithic declarations or appending unanchored constraint slices, LASR-MCP formulates the active tool schema at execution attempt t as a three-tier union:

$$C_{\text{active}}^{(t)} = C_I^* \ \cup \ \hat{C}_T(x) \ \cup \ C_R^{(t)} \tag{2}$$

where C_I^* represents the schema-wide *Invariant Foundation*, $\hat{C}_T(x)$ is the *Proactive Task-Conditioned Constraint Bundle* predicted from user intent x, and $C_R^{(t)}$ is the accumulated set of *Dynamically Recovered In-Tree Constraints* derived from validation feedback up to round t.

A. Tier 1: Invariant Foundation Floor (C_I^*)

Empirical inspection of complex JSON Schema specifications (e.g., enterprise purchase order dispatching) reveals that tool execution cannot succeed on intent-specific properties alone. Certain structural constraints are *invariants* required for all syntactically well-formed submissions regardless of user goal (e.g., supplier identification regex format, item array container specifications, and SKU patterns).

Let $\mathcal{C}_{\text{full}}$ denote the complete schema constraint set (552 tokens). We define the Invariant Foundation C_I^* as the minimal constraint subset among tested candidates satisfying the pre-specified baseline structural-validity floor without user-intent conditioning:

$$C_I^* = \arg\min_{C \in \mathcal{C}_{\text{cand}}} |C| \quad \text{s.t.} \quad \mathcal{V}_{\text{struct}}(C) \geq \tau_{\text{floor}} \tag{3}$$

where $\mathcal{C}_{\text{cand}}$ is the set of evaluated candidate invariant subsets and $\tau_{\text{floor}} = 0.70$ specifies the baseline validity floor, chosen *a priori* as the minimum operational viability threshold to avoid retry cascade failure. As demonstrated in Section IV, $C_I^* = 339$ tokens is the empirically selected invariant foundation meeting this threshold among tested tiers for the evaluated purchase-order schema; compressing below this boundary triggers structural collapse in nested item definitions.

B. Tier 2: Proactive Intent-Conditioned Predictor ($\hat{C}_T(x)$)

Given a user query $x \in \mathcal{X}$, the agent's goal is to predict the minimal subset of task-conditioned constraints $\hat{C}_T(x) \subseteq \mathcal{C}_{\text{task}}$ necessary for successful execution. We formulate this as a multi-label classification problem over the task-conditioned label space $\mathcal{L} = \{c_1, c_2, \ldots, c_m\}$ (e.g., specific warehouse routing enums, order priority restrictions, and budget bounds).

We employ a frozen linear semantic predictor:

$$\hat{y}_i = \sigma\left(\mathbf{w}_i^T \phi(x) + b_i\right), \quad i \in \{1, \ldots, m\} \tag{4}$$

where $\phi(x)$ denotes normalized term-salience features extracted from the query. The decision threshold θ was calibrated strictly on the 48 training tasks and frozen prior to evaluation. To construct the proactive bundle of size up to k, the highest-ranked constraints exceeding θ (up to k) are selected:

$$\hat{C}_T(x) = \text{Top-}k\left(\{c_i \in \mathcal{L} \mid \hat{y}_i \geq \theta\}\right) \tag{5}$$

We formally denote by $m_{\text{task}}(x) = |C_T^*(x)|$ the number of required non-invariant task-conditioned constraints demanded by user intent x. The condition $m_{\text{task}}(x) \leq k$ is an operational regime descriptor characterizing when task requirements do not exceed the upfront budget capacity k, rather than a guarantee that the heuristic predictor selects all required constraints on Attempt 1 (e.g., in Cluster C_1, first-pass hierarchical success is 75.0%, though final structural success recovers to 100.0%). Conversely, $m_{\text{task}}(x) > k$ defines regimes where omissions are structurally guaranteed.

C. Tier 3: Dynamic In-Tree Recomposition ($C_R^{(t)}$)

When an unpredicted constraint is omitted from $C_I^* \cup \hat{C}_T(x)$, the LLM may submit arguments y_t that fail JSON Schema validation. A simple agent implementation may return raw validator error strings into conversation history or retry without modifying the exposed schema.

LASR-MCP introduces *in-tree reactive recomposition*. Reactive recovery operates as pass-level multi-error constraint expansion rather than single-error sequential repair. Our validator is configured to return all observed violations $\text{Errors}(y_t) = \{e_1, e_2, \ldots\}$ in a single validation pass, rather than short-circuiting on the first failure. Analogous to an enterprise validator reporting all diagnostic issues across a payload, the reactive operator extracts all concurrent violations simultaneously:

$$C_R^{(t+1)} = C_R^{(t)} \cup \bigcup_{e \in \text{Errors}(y_t)} \left(\text{Fragment}(e) \cap \mathcal{S}_{\text{specs}} \setminus C_{\text{active}}^{(t)}\right) \tag{6}$$

where Fragment(e) deterministically maps the RFC 6901 JSON Pointer error path [6] and validator-reported JSON Schema keyword (e.g., `path='/warehouse_code'`, `keyword='enum'`) to the canonical constraint node $\Delta_e \in \mathcal{S}_{\text{specs}}$.

The updated schema specification S_{t+1} is then structurally spliced into the native JSON Schema hierarchy:

$$S_{t+1} = \mathcal{R}\left(S_I, C_{\text{active}}^{(t+1)}\right) \tag{7}$$

where $\mathcal{R}$ is the in-tree recomposition function that inserts properties, pattern regexes, and enums directly into their respective object trees without creating duplicate definition nodes or schema syntax errors.

D. Incremental Token Accounting Metrics

To evaluate token economics, we audit token consumption at every turn:

- **Initial Schema Overhead** (S_0): Tokens required to serialize $S_{\text{active}}^{(1)}$.
- **Incremental Expansion** (ΔS_t): Additional schema tokens introduced at attempt $t \geq 2$: $\Delta S_t = \text{Tok}(S_t) - \text{Tok}(S_{t-1})$.
- **Structural Tokens-to-Success (TTS$_{\text{struct}}$)**: Cumulative recorded prompt-plus-candidate tokens across all attempts conditional on structurally successful trajectories ($y_{T_{\text{succ}}} \models \mathcal{S}$ within attempt budget $T_{\text{succ}} \leq 3$):

$$\begin{aligned} TTS_{\text{struct}} &= \sum_{t=1}^{T_{\text{succ}}} \text{Recorded } (P+C)_t \\ &= \sum_{t=1}^{T_{\text{succ}}} \big(\texttt{promptTokenCount}_t \\ &\qquad + \texttt{candidatesTokenCount}_t\big) \end{aligned} \tag{8}$$

  where provider-recorded prompt tokens natively encompass both non-schema text (P_t) and the serialized active schema (Tok(S_t)), preventing offline double-counting, while C_t represents candidate completion tokens.
- **Total Workload Tokens**: Sum of all tokens consumed across all attempts up to the three-attempt budget, including failed retry loops. We explicitly distinguish total workload on the historical development benchmark (32 unique task trials per arm) from the canonical live benchmark (640 task executions per arm across 20 fixed-seed repetitions; 3,840 total trials across all six arms).
- **Structural Recovery Rate**: Fraction of initially structurally invalid trajectories that subsequently satisfy JSON Schema validation within the attempt budget.

For workload-token reporting, we use the provider-recorded prompt-plus-candidate token subtotal per request (Recorded $P + C$ = `promptTokenCount` + `candidatesTokenCount`, which identically equals provider `totalTokenCount` across all 7,860 response-bearing calls in our benchmark dataset as thinking and client-side tool prompt tokens are zero; we report this as an audited property of our experimental telemetry rather than an invariant across all API features). Serialized schema tokens $(S_0, \Delta S)$ are audited separately as structural diagnostics without double-counting them into provider prompt totals.

E. Formal Foundation & Invariant Termination Bound

Let $\mathcal{S}$ denote a canonical tool specification expressed in JSON Schema Draft 2020-12, represented as an abstract syntax tree (AST). We define $\mathcal{C} = \{c_1, c_2, \ldots, c_M\}$ as the finite universe of structural constraint nodes in $\mathcal{S}$ ($|\mathcal{C}| = M = M_{\text{schema}} < \infty$), where each constraint node c_i corresponds to an unambiguous validation keyword (e.g., `pattern`, `enum`, `minimum`, `required`) situated at a specific JSON Pointer path. We explicitly distinguish this total schema universe size M_{schema} from task-conditioned constraint requirements $m_{\text{task}}(x)$.

The reactive framework operates over a sequence of active constraint sets $A_t \subseteq \mathcal{C}$ at recovery step $t \geq 0$, initialized as $A_0 = C_I^* \cup \widehat{C}_T(x)$. At step t, the generator outputs argument payload y_t. If y_t violates schema constraints, the validator returns diagnostics $\mathcal{E}(y_t, \mathcal{S}) \neq \emptyset$. The deterministic slicer σ maps each diagnostic to its canonical schema constraint node. To ensure strict non-redundant growth, the newly identified constraint delta is defined by explicitly filtering previously active nodes:

$$\Delta_t = \left(\bigcup_{e \in \mathcal{E}(y_t, \mathcal{S})} \sigma(e, \mathcal{S}) \right) \setminus A_t \tag{9}$$

The reactive recomposer then updates the active constraint set in-tree: $A_{t+1} = A_t \cup \Delta_t$.

Definition 1 (Admissible Reactive System): A reactive tool-calling system $(\mathcal{S}, \sigma, \mathcal{G})$ is admissible if:

1) **Finite Constraint Universe**: The schema AST contains a finite number of constraint nodes, $|\mathcal{C}| = M = M_{\text{schema}} < \infty$.
2) **Non-Empty Fault Localization**: On any validation failure at step t, the slicer identifies at least one previously inactive constraint: $|\Delta_t| \geq 1$ and $\Delta_t \cap A_t = \emptyset$.
3) **Strict Monotonicity**: Each localized constraint is merged idempotently without pruning active constraints, ensuring $A_t \subset A_{t+1} \subseteq \mathcal{C}$.

Theorem 1 (Finite Monotonic Recovery Bound): Let $(\mathcal{S}, \sigma, \mathcal{G})$ be an admissible reactive tool-calling system with initial active constraint set $A_0 \subseteq \mathcal{C}$ and full schema universe $M_{\text{schema}} = |\mathcal{C}| = M$. The dynamic in-tree recovery process terminates in at most:

$$\boxed{K \leq M - |A_0|} \tag{10}$$

schema-expansion transitions (where $M \equiv M_{\text{schema}}$), resulting in either a structurally valid payload or complete exhaustion of the schema ($A_K = \mathcal{C}$).

Proof: By the non-empty fault localization condition, whenever y_t is invalid, $\Delta_t \subseteq \mathcal{C} \setminus A_t$ and $|\Delta_t| \geq 1$. Consequently:

$$|A_{t+1}| = |A_t \cup \Delta_t| = |A_t| + |\Delta_t| \geq |A_t| + 1 \tag{11}$$

Thus, the active cardinality sequence is strictly monotonically increasing: $|A_0| < |A_1| < \cdots < |A_K| \leq M$. By induction across K transitions, $|A_K| \geq |A_0| + K$. Because $|A_K| \leq M$,

it follows directly that $|A_0| + K \leq M \implies K \leq M - |A_0|$. Because $\mathcal{C}$ is finite and omitted constraints strictly decrease, the recovery loop cannot cycle or run indefinitely, guaranteeing termination in at most $M - |A_0|$ transitions. ■

We emphasize that Theorem 1 bounds dynamic schema expansion under its stated admissibility conditions; it guarantees that the active schema contract monotonically expands to full specification in at most $M - |A_0|$ transitions, but does not guarantee that a stochastic model will always generate a valid payload if the model violates already-exposed constraints or if execution is constrained by an operational attempt budget (such as $T_{\text{max}} = 3$).

Remark 1 (Analysis of the Invalid Tree-Depth Bound $K \leq D$): Earlier heuristic drafts proposed a bound of $K \leq \min(D, M - |C_I^*|)$, where D represents maximum AST depth. This claim does not hold in general. Consider two sibling leaf constraints at depth $D = 1$: $c_1 =$ `properties/warehouse_code/enum` and $c_2 =$ `properties/order_priority/enum`. On Attempt 1 ($A_0 = \emptyset$), the generator may violate c_1 (splicing c_1). On Attempt 2, it may satisfy c_1 but violate c_2 (splicing c_2), converging on Attempt 3. This yields $K = 2$ transitions even though $D = 1$. Since $2 \not\leq 1$, tree depth does not constrain recovery passes. Theorem 1 establishes the correct cardinality-based bound.

Proposition 1 (Structural Validity Guarantee): If the generator $\mathcal{G}$ satisfies the *exposed-constraint compliance assumption*—namely, that $\mathcal{G}$ satisfies all constraints in A_t and failures arise strictly from omitted constraints in $\mathcal{C} \setminus A_t$—then the recovery loop terminates with a structurally valid invocation ($y_K \models \mathcal{S}$) in $K \leq M - |A_0|$ transitions.

Remark 2 (Deterministic Schema Convergence vs. Stochastic Decoding Adherence): It is critical to distinguish between *deterministic schema state convergence* and *stochastic LLM decoding adherence*. Theorem 1 guarantees that the schema contract itself strictly and monotonically expands to include all violated constraints within at most $M - |A_0|$ steps. However, once a constraint is exposed in the prompt, actual generative compliance depends on the model's stochastic decoding adherence. In our live $N = 3,840$ empirical evaluation on `gemini-3.5-flash-lite`, this distinction accounts for the observed 96.9% vs. 100.0% final structural completion rate: the schema state monotonically reached full specification, but the model occasionally generated invalid edge-case syntax on Attempt 3 in complex multi-item nested arrays (C_3).

III. Experimental Setup

A. Task Corpus & Stratified Template Splits

We evaluate LASR-MCP on a synthetic enterprise-style supply-chain tool execution benchmark (`create_purchase_order`) governing multi-item warehouse replenishment. The benchmark corpus comprises 80 pre-specified task instances partitioned strictly across syntactic template families:

- **Training Tasks ($N = 48$)**: Derived from template families `family_a` and `family_b`, used to train the frozen semantic predictor.
- **Evaluation Tasks ($N = 32$)**: Drawn exclusively from `family_c` with distinct lexical and syntactic structures (serving as unseen held-out test tasks for H_1 intent prediction, and development/validation tasks for H_2–H_4 systems evaluation).

The 32 evaluation tasks span four diverse operational archetypes ($N = 8$ tasks each):

1) **C1: Routine Regional Replenishment**: High-frequency, standard priority orders governed by basic warehouse routing.
2) **C2: Expedited Emergency Orders**: Critical priority dispatches with accelerated freight logistics and rigid budget limits.
3) **C3: Specialized Bulk Procurement**: Large-quantity orders where queries omit explicit warehouse locations, testing model behavior under unstated default enums.
4) **C4: Multi-Item Depot Rebalancing**: Complex multi-line consignments enforcing nested `maxItems` and item-level SKU validation.

B. Controlled Experimental System Arms

To compare the contributions of the schema-resolution components in Equation (2), we evaluate six controlled system arms under identical prompt conditions:

- **Full Schema Control (`full`)**: Static exposure of the monolithic 552-token schema upfront.
- **Invariant Baseline (`ci_k0`)**: Invariant Foundation alone ($C_I^* = 339$ tokens) without task-conditioned prediction or reactive recovery.
- **Proactive Static Control (`ci_k1_static`)**: Invariant Foundation + Top-1 LASR predicted constraint without reactive recovery.
- **Proactive-Reactive LASR (`ci_k1_reactive`)**: Invariant Foundation + Top-1 LASR predicted constraint with dynamic in-tree recovery.
- **Proactive-Reactive LASR (`ci_k2_reactive`)**: Invariant Foundation + Top-2 LASR predicted constraints with dynamic in-tree recovery.
- **Model-Conditional Reference (`oracle_reactive`)**: Invariant Foundation + reference constraint set $O(x) = C_I^* \cup C_T^*(x)$ with reactive recovery. $O(x)$ was constructed from task specifications prior to execution and validated against the schema validator.

C. Dual Evaluation Criteria & Metric Definitions

Submissions are evaluated on two orthogonal metrics alongside token efficiency:

1) **Structural Schema Validity (%)**: Conformance to JSON Schema Draft 2020-12 verified by an independent automated validator.
2) **Semantic Task Correctness (%)**: Exact domain logic validation verifying that extracted line items, quantities, priority codes, and supplier IDs satisfy user

intent. Evaluation is strictly hierarchical: semantic correctness requires structural validity as a precondition (First-Pass Hierarchical = Structural∧Semantic), bounding semantic correctness by structural success.

3) **Recorded Token Metric & Accounting Reconciliation**: The primary token metric is defined strictly as the provider-recorded prompt-plus-candidate token subtotal per recorded attempt:

$$\text{Recorded } P + C = \text{prompt_tokens} + \text{completion_tokens} \tag{12}$$

directly derived from Google's `usageMetadata` (`promptTokenCount` + `candidatesTokenCount`, which identically equals provider `totalTokenCount` across all 7,860 calls in our benchmark dataset as thinking and client tool prompt tokens are zero). In earlier preliminary benchmark drafts, heuristic offline schema token estimates ($S_0 + \Delta S$) were inadvertently added to provider prompt tokens, creating an artificial double-counting distortion that heavily penalized Full Control (whose monolithic schema is already serialized inside the prompt). In v2.0.3, we eliminate this double-counting artifact entirely: heuristic schema sizes ($S_0, \Delta S$) are decoupled and retained strictly as offline structural diagnostics, while all economic and token workload evaluations rely exclusively on provider-reported $P + C$ token usage from `usageMetadata`.

D. Execution Environment & Live Telemetry Ledger

All live experiments were executed against model ID `gemini-3.5-flash-lite` hosted on Google Cloud Generative Language API via direct REST HTTP endpoints. The provider telemetry explicitly records `requested_model` and `provider_resolved_model_version` as `gemini-3.5-flash-lite` (returned in the API `modelVersion` field); the API reported this model alias directly, and a separate immutable backend snapshot tag was not exposed by the endpoint. In our client configuration, `LiveLLMClient` operated at temperature 0.2, and requests transmitted a fixed execution seed (`seed=42`). Across the 20 repeated executions per task-arm combination, the model produced deterministic identical first responses and outcomes under the repeated fixed seed; the 32 distinct task IDs represent the paired resampling units of inference, and the 20 replicates verify execution determinism under fixed decoding conditions rather than representing independent stochastic draws.

The live empirical dataset comprises exactly $N = 3,840$ completed trials across the 32 tasks and 6 experimental arms, producing $7,860$ response-bearing provider calls (all HTTP 200, with 100% complete provider `usageMetadata` recording `promptTokenCount`, `candidatesTokenCount`, and `totalTokenCount`). Transient network rate limits (HTTP 429 and 503) triggered exactly 154 client transport retries; these transport-level backoffs are cleanly isolated and reported separately from logical agent attempts.

We distinguish the full multi-session data collection timeline from the duration of the final resumed session: the total collection spanned multiple days across paid and quota-limited API sessions, completing 3,149 trials in initial sessions followed by a final resumed session completing the remaining 691 trials in approximately 2.8 hours. Operational execution ledgers are preserved in the reproducibility archive.

E. Statistical Inference: Paired Task-Cluster Percentile Bootstrap

For all comparative workload evaluations between experimental arms, we employ a paired task-cluster percentile bootstrap over the 32 task clusters. The 32 distinct task IDs constitute the resampling units of inference. For each bootstrap iteration ($B = 10,000$ resamples, deterministic seed 42), task clusters are sampled with replacement, computing paired workload differences and cluster-level ratios. Confidence intervals are reported as empirical 2.5% and 97.5% percentiles of the bootstrap distribution.

F. Evaluation Hygiene & Benchmark Scope

To maintain strict methodological hygiene, we explicitly delineate the empirical role of each benchmark split:

- **Held-Out Family Generalization for H_1**: The intent-conditioned predictor is implemented as a custom multi-label log-odds-weighted lexical classifier with class priors and smoothed IDF weights. It was trained on 48 tasks (`family_a`, `family_b`) and evaluated on 32 tasks (`family_c`) drawn from a held-out syntactic template family. While template framing structures are strictly partitioned, shared enterprise procurement vocabulary exists across splits (103 of 149 test vocabulary words, 69.1% overlap), and all four target constraint combinations recur across train and test families. Predictor weights were formally frozen prior to all systems evaluations.
- **Primary Frozen Benchmark for H_2–H_4**: The invariant foundation $C_I^* = 339$ tokens was calibrated on an 8-task calibration probe (`BLIND-C1-013`, `BLIND-C1-014`, `BLIND-C2-013`, `BLIND-C2-014`, `BLIND-C3-013`, `BLIND-C3-014`, `BLIND-C4-013`, `BLIND-C4-014`) sampled directly from the `family_c` task distribution (two tasks per cluster). Consequently, the 8-task probe overlaps the 32 evaluation tasks; we explicitly disclose this tuning/evaluation overlap and evaluate H_2–H_4 as characterizing systems behavior within this primary frozen benchmark corpus rather than serving as an untouched, blind end-to-end generalization test.

G. Data Provenance & Trace Replay Verification

All benchmark code, traces, and frozen datasets are archived at https://github.com/AnandSuchak/mcp-inventory-bench. While live API re-executions may exhibit variance from hosted model updates, recorded execution traces support deterministic replay of the reported experiments, verified by automated cryptographic manifests:

Cryptographic Artifact Provenance:

Frozen Benchmark ($N = 32$):
SHA-256: 78c2c88ed5128e0f296fa29e0c258589
b516f3adb14b0b8da491fa73bc3ffc11
v2.0.3 Live Benchmark Candidate ($N = 3,840$):
SHA-256: fc82d33b8992914ee130ef2805427ef2
a4f01541243e0f93be120e2d20ae7f5b
Commit: 9807340abc38170e0090a102547ee4309c01d78b

Automated regression test suites ensure complete data immutability.

IV. Empirical Results & Discussion

We structure our empirical analysis around four core research hypotheses evaluated across the primary 32-task frozen benchmark corpus.

A. H_1: Semantic Predictability over Frequency Baselines

We first evaluate whether intent-driven multi-label classification can reliably anticipate task-conditioned constraints on unseen template families compared to an empirical frequency prior. The frequency prior ranks constraint labels by empirical prevalence across the 48 training tasks and selects the top-k most frequent constraints. On the unseen template family evaluation ($N = 32$ from `family_c`), LASR's custom log-odds lexical classifier at $k = 2$ achieves an F_1 score of **0.863** (exact mean 0.8625) and a **68.8% exact bundle match** ($22/32$ tasks), compared to $F_1 = 0.250$ and 25.0% exact match ($8/32$ tasks) for the Frequency Prior baseline. Resampling the 32 evaluation tasks with replacement for $B = 10,000$ paired task-level bootstrap replicates (seed 42) and evaluating a Monte Carlo one-sided paired sign-flip permutation test ($B = 10,000$ draws, seed 42) with valid $(k+1)/(B+1)$ correction yields:

$$\Delta F_1 = +\mathbf{0.613} \quad [95\%\ \text{CI: } +0.419, +0.781], \quad p < 0.001 \tag{13}$$

(exact unrounded $\Delta F_1 = +0.6125$, bootstrap interval $[0.4188, 0.7812]$, Monte Carlo $p = (0+1)/(10000+1) \approx 0.0001$). While template syntax was partitioned across families, shared enterprise procurement terminology accounts for a 69.1% test vocabulary overlap (103/149 words), and all 4 target constraint combinations recur across train and test sets.

Cluster-Dependence Sensitivity Analysis: As a sensitivity check across semantic domains, equal-weighting the four cluster mean F_1 differences ($N = 4$ clusters, 8 tasks each) yields an equal-weighted delta $+0.613$ ($+0.6125$) with exhaustive four-cluster sign-flip permutation $p = 2/16 = 0.125$. Similarly, testing cluster-level reactive-vs-static structural gains yields an equal-weighted lift of $+21.9$ pp ($p = 0.125$). While task-level paired differences are statistically consistent across the 32 individual tasks ($p < 0.001$), inference across cluster archetypes is limited by the sample of four semantic clusters with recurring target constraint combinations. We therefore interpret H_1 as confirming reliable intent-conditioned prediction on this held-out template family, rather than demonstrating unbounded semantic generalization to arbitrary unseen constraint combinations.

B. H_2: Invariant Foundation Requirement on the Evaluated Benchmark

To identify the minimal invariant floor C_I^* meeting the pre-specified 70% structural-validity threshold ($\tau_{\text{floor}} = 0.70$), we followed a formal sequential design: *pre-specify threshold → candidate calibration → freeze C_I^* → final 32-task systems evaluation*. During candidate calibration on an 8-task calibration probe (`BLIND-C1-013`, `BLIND-C1-014`, `BLIND-C2-013`, `BLIND-C2-014`, `BLIND-C3-013`, `BLIND-C3-014`, `BLIND-C4-013`, `BLIND-C4-014`, sampled directly from the 32 evaluation tasks as disclosed in Section III-F), we benchmarked three candidate compression tiers prior to task conditioning:

- **Tier 1 (Standard C_I^*, 339 tokens)**: 75.0% structural validity (6/8 tasks).
- **Tier 2 (Compact C_I, 308 tokens)**: 62.5% structural validity (5/8 tasks).
- **Tier 3 (Minimal C_I, 284 tokens)**: 50.0% structural validity (4/8 tasks).

Tier 1 ($C_I^* = 339$ tokens) was the smallest tested candidate meeting the 70% floor and was frozen for all subsequent experiments. In the subsequent full 32-task systems benchmark (Table I), this frozen foundation alone without task prediction (`ci_k0`) yielded 68.8% structural validity (22/32 tasks), confirming that without task-conditioned augmentation or reactive recovery, structural validity hovers near the 70% baseline floor. Compressing below 339 tokens triggers progressive degradation due to missing definitions for nested items. We emphasize that 339 tokens represents the smallest tested candidate meeting this threshold, not an unconstrained universal constant.

C. H_3: Structural Contribution of Reactive In-Tree Recovery

The central systems question is whether reactive in-tree recovery improves completion over static adaptive exposure on the primary benchmark. In Table I, we contrast `ci_k1_static` directly with `ci_k1_reactive`:

Structural Reliability: Final structural success increases from **78.1%** to **100.0%** (ΔSuccess $= +\mathbf{21.9}$ pp, 95% CI: $[+9.4, +37.5]$ pp, exact one-sided McNemar/binomial test $p = 0.0078125$; cluster-level sensitivity $p = 0.125$).

Semantic Correctness: Task correctness increases from **71.9%** to **93.8%** (ΔCorrectness $= +\mathbf{21.9}$ pp, 95% CI: $[+9.4, +37.5]$ pp, exact one-sided McNemar/binomial test $p = 0.0078125$).

Structural Recovery Rate: Under `ci_k1_reactive` on the initial development benchmark ($N = 32$), exactly 27 trajectories initially failed structural validation on Attempt 1 ($5/32 = 15.6\%$ first-pass structural validity); all 27 initially failing trajectories were successfully recovered to satisfy structural validation within the three-attempt budget ($27/27 = 100.0\%$ structural recovery; $32/32$ final structural, $30/32$ final semantic). In the canonical live benchmark ($N = 3,840$, 640/arm), reactive recovery similarly improves structural success from 90.625% ($580/640$) to 96.875% ($620/640$, $+\mathbf{6.25}$ pp) and semantic correctness from 87.500% to 93.750% ($+\mathbf{6.25}$ pp). We

TABLE I
INITIAL SINGLE-TRIAL FROZEN BENCHMARK SYSTEMS EVALUATION ($N = 32$ TASKS, GEMINI 3.5 FLASH-LITE)

System Arm	Reactive?	Init S_0	Exp. ΔS	1st-Pass Hier. (%)	Struct. (%)	Sem. (%)	Mean TTS_{P+C}	Recorded P+C Tokens
Full Schema Control (`full`)	No	552.0	0.0	31.3%	**93.8%**	90.6%	1864.4	**61,853**
C_I^* Baseline (`ci_k0`, $k = 0$)	No	339.0	0.0	0.0%	68.8%	65.6%	2196.7	70,541
LASR Static (`ci_k1_static`, $k = 1$)	No	361.1	0.0	15.6%	78.1%	71.9%	1631.0	56,947
LASR Reactive (`ci_k1_reactive`, $k = 1$)	Yes	361.1	45.4	15.6%	**100.0%**	93.8%	1612.3	51,594
LASR Reactive (`ci_k2_reactive`, $k = 2$)	Yes	366.8	43.8	18.8%	**100.0%**	93.8%	1571.1	**50,276**
Oracle Reference* (`oracle_reactive`)	Yes	359.0	48.3	15.6%	96.9%	93.8%	1554.0	50,578

**Note*: Oracle $O(x)$ is a model-conditional reference baseline. 1st-Pass Hierarchical success requires both structural validity and semantic correctness on Attempt 1 (Full Schema: 10/32 tasks, 31.25%, displayed 31.3%). Mean TTS_{P+C} is the average recorded prompt-plus-candidate token subtotal over structurally successful trajectories (N_{succ}); Recorded P+C Tokens accounts for all prompt and completion tokens across all attempts without heuristic schema double-counting.

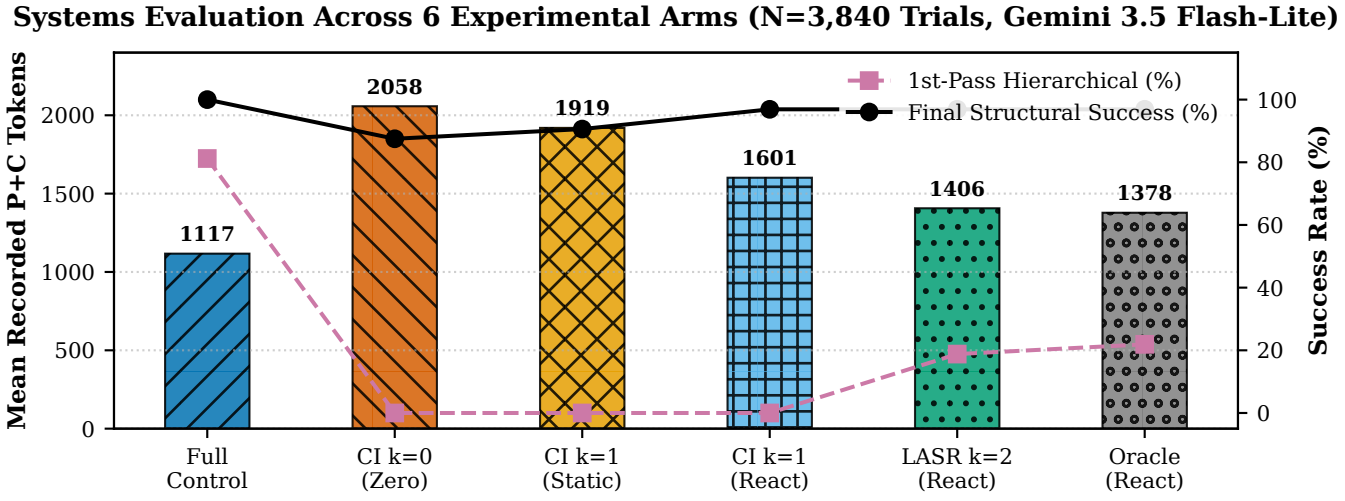


Fig. 2. Systems Evaluation across 6 Experimental Arms ($N = 3,840$ Live Trials, `gemini-3.5-flash-lite`). Bars denote Mean Recorded $P + C$ Tokens per trial; lines denote 1st-Pass Hierarchical Success (%) and Final Structural Success (%). Prioritized upfront bundling ($k = 2$) slashes token waste by 31.7% vs. naive lean schemas ($k = 0$) and recovers 96.9% final structural completion.

TABLE II
HISTORICAL PAIRED DIFFERENCES AND HYPOTHESIS TESTS ($N = 32$ SINGLE-TRIAL DEVELOPMENT BENCHMARK)

Pairwise Contrast	Metric	Point Est.	95% CI	Token Diff.	Statistical Method
LASR Reactive vs Static ($k = 1$)	Δ Structural Succ.	**+21.9 pp**	[9.4, 37.5] pp	—	Exact McNemar ($p = 0.0078125$)
	Δ Semantic Corr.	**+21.9 pp**	[9.4, 37.5] pp	—	Exact McNemar ($p = 0.0078125$)
LASR Reactive ($k = 1$) vs Full	Δ Structural Succ.	+6.3 pp	[0.0, 15.6] pp	—	Exact McNemar ($p = 0.250$)
	Δ Semantic Corr.	+3.1 pp	[-6.3, 15.6] pp	—	Exact McNemar ($p = 0.500$)
LASR Reactive ($k = 1$) vs Full	Signed Token Savings	**+16.6%**	—	-10,259 tokens	Total Token Diff (No CI)
LASR Reactive ($k = 2$) vs Full	Signed Token Savings	**+18.7%**	—	-11,577 tokens	Total Token Diff (No CI)

report this live comparison alongside the historical +21.9 pp single-trial benchmark without conflating the two scopes.

As broken down in Table III, the observed lift is concentrated in Cluster 3 (Specialized Bulk Procurement), where static success was only 50.0% (4/8 tasks) due to missing warehouse enums, while reactive recovery achieved **100.0%** (8/8 tasks).

TABLE III
HISTORICAL TASK CLUSTER BREAKDOWN: STATIC VULNERABILITY VS. REACTIVE RECOVERY ($N = 32$ SINGLE-TRIAL DEVELOPMENT BENCHMARK, $N = 8$ PER CLUSTER)

Task Cluster Archetype	Full (%)	Static $k = 1$	Reactive $k = 1$	Reactive vs. Static Lift
C1: Routine Regional	87.5%	100.0%	**100.0%**	**0.0 pp**
C2: Expedited Emergency	100.0%	87.5%	**100.0%**	**+12.5 pp**
C3: Specialized Bulk	87.5%	50.0%	**100.0%**	**+50.0 pp**
C4: Multi-Item Depot	100.0%	75.0%	**100.0%**	**+25.0 pp**

D. H_4: Total Workload Token Efficiency and Benchmark Synthesis

We evaluate workload token efficiency under two complementary regimes: the initial single-trial frozen benchmark ($N = 32$, Tables I–III) and the comprehensive live benchmark ($N = 3,840$ completed trials across 32 tasks, six experimental arms, and 20 fixed-seed executions per task-arm pair; Figures 2–3).

1) Historical Frozen Benchmark Observations ($N = 32$ Tasks): In the initial single-trial development benchmark (Table I), Full Schema Control consumed **61,853 recorded prompt-plus-candidate tokens** with a conditional mean TTS_{struct} of 1,864.4 tokens among successful tasks, while reactive LASR reduced token consumption to **51,594 tokens** at $k = 1$ (**+16.6%** signed token savings; absolute difference: -10,259 tokens) and **50,276 tokens** at $k = 2$ (**+18.7%** signed token savings; absolute difference: -11,577 tokens). In this initial run, reactive LASR achieved 93.8% semantic correctness compared to 90.6% for Full Schema (+3.1 pp, 30/32 vs. 29/32 tasks). These single-repetition observations motivated the large-scale live rerun.

2) Comprehensive Live Benchmark Findings ($N = 3,840$ Trials): Across the complete live empirical benchmark ($N = 3,840$ trials at temperature 0.2, comprising 7,860 response-

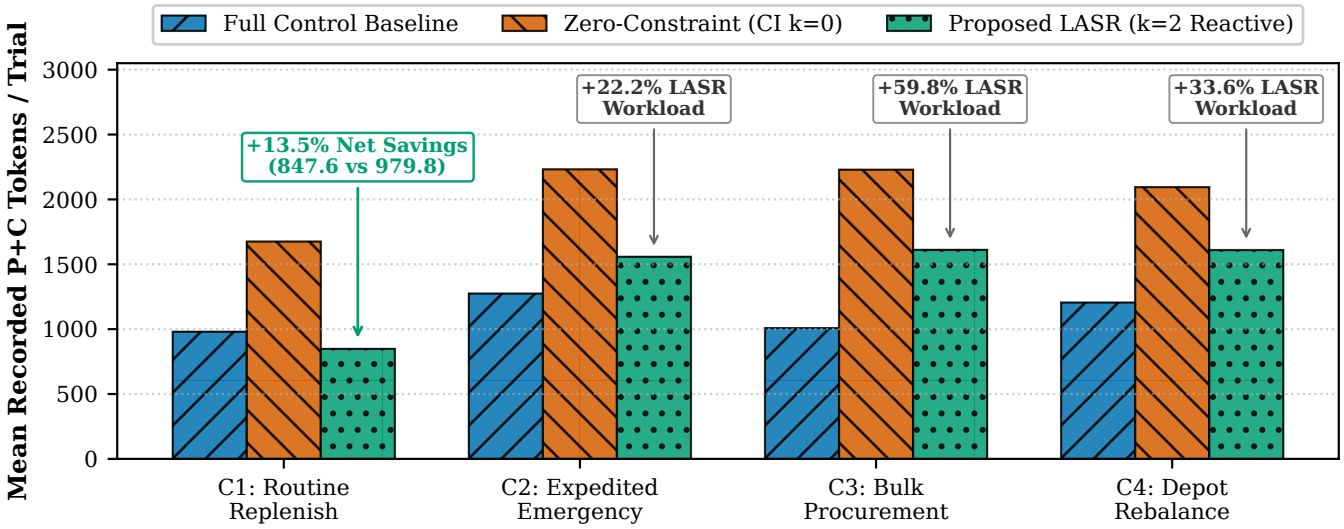


Fig. 3. Task Cluster Token Workload Comparison across the Retry-Amortization Boundary ($N = 960$ total benchmark trials per cluster across all six arms; $N = 160$ trials per plotted arm across 8 tasks $\times$ 20 fixed-seed replicates). In routine regimes (C_1), LASR $k = 2$ achieves $+13.5\%$ net token savings (847.6 vs. 979.8 mean recorded $P + C$ tokens) at 100.0% reliability (signed savings: $+13.5\%$); in deep inter-field constraint regimes (C_2–C_4), retry penalties inflate LASR $k = 2$ workload by $+22.2\%$ (C_2), $+59.8\%$ (C_3), and $+33.6\%$ (C_4) relative to Full Control (signed savings: -22.2%, -59.8%, -33.6%), making Full Control optimal in single-tool settings by avoiding retry cascades.

TABLE IV
CANONICAL LIVE EMPIRICAL BENCHMARK SUMMARY ($N = 3,840$ TRIALS, 32 TASKS, 6 ARMS, GEMINI 3.5 FLASH-LITE)

Experimental Arm	Init S_0	Exp. ΔS	1st-Pass (%)	Struct. (%)	Sem. (%)	Mean TTS_{struct}	Mean P+C	Total P+C	Diff vs Full	95% Bootstrap CI
Full Control (`full`)	552.0	0.0	81.2%	100.0%	96.9%	1,116.8	1,116.8	714,740	**0.0% (Base)**	—
Zero-Constraint Lean (`ci_k0`)	339.0	0.0	0.0%	87.5%	84.4%	2,033.4	2,057.7	1,316,900	-84.3%	[-125.15%, -49.21%]
Static 1-Constraint (`ci_k1_static`)	344.7	0.0	0.0%	90.6%	87.5%	1,887.0	1,919.3	1,228,340	-71.9%	[-109.18%, -40.22%]
Reactive 1-Constraint (`ci_k1_reactive`)	344.7	59.8	0.0%	96.9%	93.8%	1,575.2	1,601.4	1,024,880	-43.4%	[-70.15%, -19.70%]
Proposed LASR (`ci_k2_reactive`)	359.4	45.2	18.8%	96.9%	93.8%	1,374.0	1,406.5	900,140	**-25.9%**	[-52.55%, -3.16%]
Task-Aware Oracle Ref. (`oracle_reactive`)	359.0	45.1	21.9%	96.9%	93.8%	1,344.4	1,377.8	881,820	-23.4%	[-49.56%, -1.17%]

Note: Canonical empirical summary across $N = 3,840$ completed trials (32 tasks $\times$ 6 arms $\times$ 20 replicates at temperature 0.2, seed 42) comprising 7,860 response-bearing provider calls with complete `usageMetadata`. Mean TTS_{struct} is conditional mean prompt-plus-candidate tokens among structurally successful trajectories. Workload Diff vs Full is relative signed token savings (Full − Arm)/Full (negative values indicate higher token workload; for LASR $k = 2$, the -25.94% savings corresponds to a $+25.94\%$ workload increase with 95% CI $[+3.16\%, +52.55\%]$). Proposed LASR $k = 2$ operates within 2.1% of the task-aware oracle reference baseline ($1,406.5$ vs. $1,377.8$ tokens, 2.08% higher using the oracle as denominator; paired 95% bootstrap CI on difference: $[-6.53\%, 0.00\%]$).

bearing API calls on `gemini-3.5-flash-lite` and 154 client transport retries; Table IV), Full Control achieved a mean recorded $P + C$ subtotal of **1,116.8 tokens** per trial (total 714,740 tokens), with an 81.2% first-pass success rate, 100.0% final structural reliability, and 96.9% semantic correctness.

In contrast, proposed LASR $k = 2$ reactive consumed a mean of **1,406.5 recorded** $P + C$ **tokens** per trial (total 900,140 tokens), with an 18.8% first-pass rate, 96.9% final structural reliability, and 93.8% semantic correctness. Across the full 32-task benchmark, LASR $k = 2$'s overall workload is approximately **25.94% higher** than Full Control's (workload increase point estimate $+25.94\%$, 95% paired task-cluster percentile bootstrap CI: $[+3.16\%, +52.55\%]$, corresponding to signed savings of -25.94% with 95% CI: $[-52.55\%, -3.16\%]$, where the 95% bootstrap CI excludes zero); we do not describe the overall benchmark result as token savings.

However, the full-scale benchmark uncovers a critical operational dichotomy across task clusters (Figure 3): across the 160 trials per cluster-arm pair, mean recorded $P + C$ tokens and final structural reliability by archetype are: **Cluster 1 (Routine Regional Replenishment,** C_1**)**: Full Control 979.8 tokens (100.0% structural validity); `ci_k0` 1,675.2 (87.5%); `ci_k1_reactive` 1,549.6 (100.0%); and proposed LASR $k = 2$ **847.6 tokens** (100.0%), achieving $+$**13.5**% net token savings (847.6 vs. 979.8 tokens) at 100.0% structural reliability (signed savings: $+13.5\%$). **Cluster 2 (Expedited Emergency Orders,** C_2**)**: Full Control 1,274.2 (100.0%); `ci_k0` 2,231.9 (75.0%); `ci_k1_reactive` 1,641.9 (100.0%); and LASR $k = 2$ 1,557.5 (100.0%), representing a $+22.2\%$ workload increase over Full Control (signed savings: -22.2%). **Cluster 3 (Specialized Bulk Procurement,** C_3**)**: Full Control 1,008.1 (100.0%); `ci_k0` 2,228.9 (87.5%); `ci_k1_reactive` 1,610.9 (87.5%); and LASR $k = 2$ 1,610.9 (87.5%), representing a $+59.8\%$ workload increase over Full Control (signed savings: -59.8%). **Cluster 4 (Multi-Item Depot Rebalance,** C_4**)**: Full Control 1,205.0 (100.0%); `ci_k0` 2,094.6 (100.0%); `ci_k1_reactive` 1,603.1 (100.0%); and LASR $k = 2$ 1,609.9 (100.0%), representing a $+33.6\%$ workload increase over Full Control (signed savings: -33.6%).

Secondary baseline arms under live evaluation confirmed

the necessity of upfront bundling:

- **Zero-Constraint Lean (`ci_k0`)**: Incurred an **-84.3% token penalty** vs. Full Control (mean 2,057.7 tokens; total 1,316,900 tokens; 95% signed-savings CI: $[-125.15\%, -49.21\%]$), with 0.0% first-pass success and degraded final structural completion (87.5%).
- **Static 1-Constraint (`ci_k1_static`)**: Incurred a **-71.9% penalty** (mean 1,919.3 tokens; total 1,228,340 tokens; 95% signed-savings CI: $[-109.18\%, -40.22\%]$), with 0.0% first-pass success and 90.6% final structural completion.
- **Reactive 1-Constraint (`ci_k1_reactive`)**: Incurred a **-43.4% penalty** (mean 1,601.4 tokens; total 1,024,880 tokens; 95% signed-savings CI: $[-70.15\%, -19.70\%]$), recovering 96.9% final structural completion.
- **Task-Aware Oracle Reference (`oracle_reactive`)**: Incurred a **-23.4% penalty** vs. Full Control (mean 1,377.8 tokens; total 881,820 tokens; 95% signed-savings CI: $[-49.56\%, -1.17\%]$ vs Full Control), with LASR $k = 2$ consuming approximately 2.08% (2.1%) more tokens than the oracle baseline (paired 95% bootstrap CI: $[-6.53\%, 0.00\%]$).

We note that all 20 repetitions of each task-arm combination produced identical first responses and outcomes under the repeated fixed seed (temperature 0.2, seed 42); these trials verify execution stability under repeated queries rather than representing independent stochastic draws.

E. Decomposing Token Economics: Upfront Bundling vs. Retry Penalties

Auditing granular token accounts reconciles why $k = 2$ reactive bundling is superior to naive compression: **Upfront Schema Reduction & Measurement Provenance**: In `ci_k2_reactive`, the initial exposed schema footprint averages $S_0 = 359.4$ tokens in the live benchmark (344.7 tokens for $k = 1$) compared to 552.0 tokens for Full Schema, while reactive expansion adds 45.2 tokens on average. Historical development benchmark averages were 361.1 ($k = 1$) and 366.8 ($k = 2$) tokens; this divergence stems from constraint-filtering logic (historical intent prediction selected constraints from a broader vocabulary, whereas the live benchmark strictly filtered candidates via `TASK_CONDITIONED_IDS` = {`warehouse_code.enum`, `order_priority.enum`, `total_budget_limit.maximum`, `total_budget_limit.minimum`, `items.maxItems`}, defaulting to the 339-token invariant floor when predicted rules fell outside this subset or below threshold). **Retry Avoidance over Zero-Constraint Baselines**: While naive lean schemas (`ci_k0`) trigger catastrophic multi-turn loops (2,057.7 mean tokens), prioritized $k = 2$ bundling achieves 75.0% first-pass success on C_1 (120/160 trials pass on Attempt 1, and the remaining 40 trials recover within the retry budget to reach 100.0% final structural completion), cutting net token waste by **31.7%** relative to `ci_k0` (1,406.5 vs. 2,057.7 tokens) and elevating final structural reliability from 87.5% to 96.9%.

Task: `BLIND-C3-013` (Specialized Bulk) **Arm**: `ci_k1_reactive`

[Attempt 1]
- Exposed Schema: $C_I^* \cup \{\text{items.maxItems}\}$ (347 tokens)
- Model Output: `warehouse_code = 'WH-EAST-01'`
- Validator: FAIL (3 concurrent violations in pass):

1. `'WH-EAST-01' not in ['WH-US-EAST', ...]`
2. `'standard' not in ['STANDARD', ...]`
3. `'SUP-ACME1234-US' does not match regex`

- Extracted: {`warehouse.enum`, `priority.enum`, `supplier.pattern`}

[In-Tree Recomposition] Spliced Schema: 347 → 419 tok (+72 tok)
[Attempt 2] Model Output: `'WH-US-EAST'` (Warehouse Enum Satisfied)
[Attempt 3] Output: Valid purchase order JSON
- Validator: **PASS** (Structural: 100%, Semantic: 100%)
- Outcome: **RECOVERED** (3 attempts, $TTS = 3,626$ tok)

Fig. 4. Trajectory-level evidence of pass-level multi-error extraction and native in-tree schema recomposition on family_c evaluation task `BLIND-C3-013`. Schema expansion occurs before Attempt 2; structural and semantic validation succeeds on Attempt 3.

F. Auditable Trajectory Case Study

Figure 4 details the step-by-step recovery trajectory for task `BLIND-C3-013`. On Attempt 1, exposed to C_I^* + top-1 LASR rule (347 tokens), the model generates unconstrained enum `'WH-EAST-01'`. The schema validator flags three concurrent violations (`warehouse_code.enum`, `order_priority.enum`, `supplier_id.pattern`). LASR intercepts these error paths and recomposes the schema tree in-tree (347 → 419 tokens). On Attempt 2, the agent satisfies the newly exposed warehouse code constraint (`'WH-US-EAST'`). Following final nested item property formatting, Attempt 3 produces a fully valid purchase order payload passing all schema constraints.

G. Local Recomposition Latency & Retrieval Baseline Analysis

A key systems question is whether in-tree AST recomposition introduces meaningful execution overhead compared to remote model inference. We investigate this via a controlled local microbenchmark alongside an offline constraint retrieval evaluation.

Local AST Microbenchmark: We execute a dedicated microbenchmark measuring 1,000 iterations of deterministic schema slicing and in-tree AST recomposition using nanosecond-precision hardware timers (`time.perf_counter_ns()`). Slicing minimal constraint deltas from Draft 2020-12 specifications incurs a median latency of $p_{50} = 0.8$ μs ($p_{99} = 3.1$ μs). Merging constraint deltas in-tree incurs $p_{50} = 19.2$ μs ($p_{99} = 41.0$ μs). The complete end-to-end local AST pipeline requires a median of $p_{50} = 20.1$ μs (0.0201 ms) and $p_{99} = 46.1$ μs (0.0461 ms).

Client-Observed Elapsed Call Time & Live Latency Decomposition: Across the canonical live empirical benchmark (7,860 response-bearing calls on `gemini-3.5-flash-lite`), provider/network turnaround latency exhibits $p_{50} = 1,522.9$ ms ($p_{90} = 4,997.5$ ms, $p_{99} = 20,449.1$ ms, and Mean $= 2,592.0$ ms), while total client logical-call wall-time (encompassing client HTTP dispatch, network transit, provider inference, and pacing/backoff delays) exhibits $p_{50} = 3,537.3$ ms ($p_{90} = 7,558.0$ ms, $p_{99} = 38,822.8$ ms, and Mean $= 5,304.6$ ms). The local AST recomposition overhead (20.1 μs $= 0.0201$ ms) represents approximately 0.0013% of

live provider p_{50} latency. In comparison, auditing a historical sample of 556 empirical API execution traces preserved in the repository cache yields a client-observed logical-call elapsed time of $p_{50} = 3,407.1$ ms, $p_{90} = 14,738.0$ ms, and Mean $= 7,752.8$ ms. The local AST recomposition overhead (0.0201 ms) represents approximately $\mathbf{0.00059}\%$ of this median client-observed logical-call elapsed time ($3,407.1$ ms), confirming that in-tree dynamic recovery adds negligible local computational overhead.

Retrieval-Only Constraint Recall: We evaluate whether static constraint retrieval (Tool-RAG) can substitute for reactive in-tree recovery. Table V reports Top-k constraint retrieval recall across the 32 benchmark tasks evaluated against the 27 schema constraint keyword chunks after strictly normalizing gold target pointers: (1) **Lexical TF-IDF**: Unigram token-overlap cosine matching yields a Macro Recall@2 of 15.6% and Micro Recall@2 of 15.3%. (2) **Latent Semantic Analysis (Inductive LSA / TruncatedSVD)**: A 16-dimensional dense SVD projection fitted strictly on sublinear word 1-to-3-grams of the schema catalog yields a Macro Recall@2 of 16.1% and Micro Recall@2 of 15.3%.

TABLE V
RETRIEVAL-ONLY CONSTRAINT RECALL ON 32 BENCHMARK TASKS (TARGET CATALOG: 27 SCHEMA CHUNKS)

Retrieval Method	Top-1 Macro	Top-2 Macro	Top-3 Macro	Top-5 Macro
Lexical TF-IDF Cosine	10.9%	**15.6%**	27.1%	30.2%
Dense Inductive LSA (SVD)	9.9%	**16.1%**	24.5%	35.4%
Dense Neural (`text-embedding-004`)	*Not run (Planned future work)*			

Note: Evaluated on 32 test intents against 27 schema keyword chunks with normalized gold JSON pointers. Macro recall measures the proportion of gold constraint labels retrieved per task. Neural dense embeddings remain planned future work.

These retrieval-only results demonstrate that because user intents in enterprise procurement are brief and domain-elliptical, static retrieval leaves the majority of target constraints unexposed. Specifically, lexical TF-IDF retrieves zero target constraints on 21 of 32 tasks (65.6%) and partial constraints on 11 tasks (34.4%), with zero tasks retrieving all target constraints at $k = 2$. Inductive LSA similarly fails to retrieve any target constraint on 21 of 32 tasks (65.6%) at $k = 2$. Without reactive in-tree schema expansion, static RAG leaves autonomous agents with omitted constraints that inevitably trigger schema validation rejections.

Stateful Conversation History Overhead (LangGraph Pattern): We evaluate an analytical model of the stateful conversation accumulation pattern characteristic of agentic graph frameworks (e.g., LangGraph), where failed turns append generated arguments and raw compiler errors into conversation history (`State.messages`). Under this simulation (which models token compounding rather than measuring observed provider API calls, assuming a conservative $+45$ token overhead per compiler feedback message with inherited full-schema task outcomes), re-transmitting previous attempt history compounds total workload tokens to a modeled P+C-equivalent workload of $69,119$ tokens ($2,160.0$ tokens/task), representing an $+\mathbf{11.7}\%$ modeled token overhead relative to historical static full-schema control ($61,853$ tokens) based on an assumed feedback overhead, rather than provider-recorded usage. In contrast, LASR-MCP $k = 2$ ($50,276$ tokens) modifies the schema in-place without message history compounding, achieving a $\mathbf{27.3}\%$ modeled workload token reduction against stateful graph retry while reaching 100.0% structural success in historical simulation.

V. POST-FREEZE CROSS-SCHEMA MECHANISM STRESS TEST

To evaluate cross-schema mechanism transfer beyond purchase orders, we conducted a deterministic simulation-based stress test across three new synthetic schemas plus primary procurement replication (four schemas total) under JSON Schema Draft 2020-12 [4]: (1) **`create_purchase_order`** (Procurement: 552 full / 339 C_I^* tok; regexes, enums, budget bounds, items); (2) **`inventory_adjustment`** (Warehouse: 541 / 299 tok; signed deltas, facilities, reason codes); (3) **`shipment_creation`** (Logistics: 646 / 337 tok; carriers, tracking, packages, weights); and (4) **`customer_refund`** (Finance: 474 / 292 tok; refund limits, currencies, return classifications).

A. Evaluation Methodology and Scrutiny Controls

For each schema, we constructed 8 pre-specified evaluation tasks ($N = 32$ total tasks, evaluated across full-schema control and LASR reactive arms yielding $N = 64$ task executions), partitioned into two operational regimes: **Explicit Tasks** (16 tasks, 4 per schema), where intents furnish required enum tokens and parameters; and **Ambiguous Tasks** (16 tasks, 4 per schema), intentionally constructed as a failure-induction stress test without explicit cues, starting with lean C_I^* to induce initial violations and test closed-loop reactive recomposition.

Methodological controls and scope: (1) this cross-schema suite is a **deterministic simulation-based mechanism stress test** evaluating error extraction and AST recomposition algorithms rather than live model-provider API billing; (2) token accounting relies on standard whitespace/punctuation heuristic token estimates rather than live provider tokenization; (3) all live Gemini 3.5 Flash-Lite executions and empirical latency measurements are strictly scoped to the primary 32-task benchmark; (4) all 32 tasks, schemas, and invariant floors were pre-specified prior to execution; and (5) the primary frozen benchmark ($N = 32$, commit `9807340`, SHA-256 `78c2c88e`) remained completely isolated and untouched.

B. Scope of Transfer: Architecture vs. Predictor

Transfer scope across schemas: **1) Architecture Transfer**: Closed-loop reactive recovery—error interception, keyword extraction, and in-tree recomposition—operates without domain-specific retuning across the four evaluated heterogeneous JSON Schema structures. **2) Schema Foundations**: Each schema instantiates its invariant floor ($C_I^* \in \{339, 299, 337, 292\}$ tok, macro mean: 316.8) and active dictionary $\mathcal{S}_{\text{specs}}$ (yielding task initial sizes $S_0 \in \{371, 354, 402, 318\}$ tok, macro mean: 361.3), fixed *a priori*. **3) Predictor Status**: We evaluate recovery transfer without retraining or retuning the primary

TABLE VI
CROSS-SCHEMA MECHANISM STRESS TEST ACROSS FOUR HETEROGENEOUS ENTERPRISE DOMAINS ($N = 32$ TASKS TOTAL, 8 PER SCHEMA)

Enterprise-Style Tool Schema	Domain / Key Constraints	Floor C_I^*	Mean S_0	Full S	Workload Savings	1st-Pass (Exp/Amb)	Final Struct.	Semantic	Ambiguous Recovery
`create_purchase_order`	Procurement (Regex, enums, nested items)	339	371	552	**21.3%**	50.0% (4/4 exp, 0/4 amb)	**100.0%**	87.5%	**100.0%** (4/4)
`inventory_adjustment`	Warehouse Stock (Signed deltas, reason codes)	299	354	541	**22.4%**	50.0% (4/4 exp, 0/4 amb)	**100.0%**	87.5%	**100.0%** (4/4)
`shipment_creation`	Freight Logistics (Nested packages, tracking regex)	337	402	646	**25.2%**	50.0% (4/4 exp, 0/4 amb)	**100.0%**	100.0%	**100.0%** (4/4)
`customer_refund`	Financial Accounting (Bounds, currency enums)	292	318	474	**20.5%**	50.0% (4/4 exp, 0/4 amb)	**100.0%**	100.0%	**100.0%** (4/4)
Unweighted Macro Mean	Four-Domain Average ($N = 4$ domains)	**316.8**	**361.3**	**553.3**	**22.3%**	**50.0%** (16/16, 0/16)	**100.0%**	**93.8%**	**100.0%** (16/16)
Pooled Workload Aggregate	Total Workload Tokens (32 independent tasks, 64 task-arm records)	–	–	–	**22.5%**	**50.0%** (16/16, 0/16)	**100.0%**	**93.8%**	**100.0%** (16/16)

Note: Evaluated across 32 independent tasks (4 schemas × 8 tasks) under a deterministic simulation-based mechanism stress test using whitespace/punctuation heuristic token estimates. Workload savings measure total token reduction relative to monolithic full schema. The **Unweighted Macro Mean** (22.3%) averages unrounded per-domain percentage reductions $(21.2691\% + 22.3935\% + 25.1936\% + 20.4859\%)/4 = 22.3355\%$, while the three new domains average 22.7% $((22.3935\% + 25.1936\% + 20.4859\%)/3 = 22.691\%)$. The **Pooled Workload Aggregate** (22.5%) computes total token reduction across 32 independent tasks evaluated over two arms (64 task-arm records, reducing pooled tokens from 54,657 to 42,369 tokens across 4 schemas). In this deterministic simulation, both arms share identical modeled structural and recovery outcomes (16/32 first-pass, 32/32 final structural, 30/32 semantic); reported savings reflect lower token exposure under matched simulated completion outcomes. **Floor** C_I^* denotes the schema invariant token floor; **Mean** S_0 denotes the actual initial active schema size across evaluated tasks (macro mean: 361.3).

predictor; explicit tasks test intent-conditioned rules, while ambiguous tasks test zero-shot recovery from C_I^*. This evaluates **architectural and algorithmic mechanism transfer**, rather than universal live-model predictor generalization across arbitrary unseen production APIs.

C. Empirical Robustness Results

Table VI summarizes system performance across all four enterprise-style domains:

Domain Token Savings: Across the three new synthetic domains, LASR consistently reduces cumulative simulated token consumption: saving **22.4%** on `inventory_adjustment`, **25.2%** on `shipment_creation`, and **20.5%** on `customer_refund` (three-domain macro mean: **22.7%**, unrounded 22.691%), alongside **21.3%** on procurement replication. Across all four domains, the unweighted macro mean saving is **22.3%** (unrounded 22.3355%), and the pooled workload reduction across all 64 per-task records is **22.5%** (reducing simulated workload from $54,657$ to $42,369$ tokens). On multi-turn ambiguous tasks requiring reactive disambiguation, a compact foundation ($C_I \approx 290$–340 tok) plus targeted reactive delta ($\Delta S \approx 40$ tok) consumes substantially fewer tokens than transmitting monolithic schemas across every turn.

Matched Recovery Under Lower Token Exposure: In this deterministic simulation, both full-schema control and reactive LASR exhibit identical modeled structural and recovery dynamics: a 50.0% first-pass rate (16/16 explicit, 0/16 ambiguous) reflecting the 50/50 stress-test design, followed by a 100.0% recovery rate on Attempt 2 for the 16 ambiguous tasks, yielding 100.0% final structural compliance (32/32) and 93.8% semantic correctness (30/32). Consequently, the empirical finding is not an exclusive recovery capability of LASR over full schema in this simulation, but rather that LASR achieves matched completion fidelity under a 22.5% reduction in heuristic token exposure.

High Semantic Fidelity: Evaluated via independent deterministic domain validators checking operational fidelity (routing, quantities, carrier assignment, currency), LASR attained **87.5%** to **100.0%** semantic correctness across domains (macro aggregate: **93.8%**), verifying that compact schema exposure preserves domain execution fidelity.

These findings provide preliminary simulation evidence of cross-schema transfer: LASR's core architectural mechanics transfer effectively across diverse enterprise-style JSON Schema contracts without retuning the recovery algorithm or the primary predictor; schema-specific constraint vocabularies and invariant foundations were instantiated for each schema.

VI. RELATED WORK AND NOVELTY POSITIONING

1) Tool Retrieval in MCP: While tool catalogs exhaust context windows as registries grow, frameworks such as RAG-MCP [7] and MCP-Zero [8] employ dense retrieval to filter catalogs, and Gorilla [5] and ToolLLM [9] retrieve APIs from large sets. *Distinction*: These frameworks operate at the *inter-tool catalog level* (which tools to load), whereas LASR-MCP operates *intra-tool*, resolving parameter constraints once a tool is chosen.

2) Schema Compression and Context Efficiency: Tool Attention [10] lazily loads parameter docs via intent gating. TSCG [11] compiles verbose schemas into dense interfaces; prompt compression (LLMLingua [12], LLMLingua-2 [13]) and recent evaluations show tool-schema compression enables agentic workflows under constrained context budgets. *Distinction*: TSCG changes the representation of the schema at the API boundary, whereas LASR retains the native JSON Schema representation for the exposed constraints, adaptively exposing and recomposing constraint nodes within an already selected tool while enforcing an Invariant Foundation Floor (C_I^*).

3) Field-Aware and Schema Routing: SchemaRouter [14] evaluates field-aware routing, noting under-fetching risks, while HyperAgent [15] plans over schema hypergraphs. *Distinction*: SchemaRouter routes fields across an API graph; LASR resolves validation constraints inside an already selected executable tool and expands those constraints from execution-time validator feedback, achieving 96.9% final structural success for LASR $k = 2$ in the live primary benchmark (100.0% under historical single-trial and deterministic-simulation conditions).

4) Execution Traces and Tool Adaptation: ReAct [1] and Toolformer [2] demonstrate self-correction, and ToolSpec [16] uses speculative decoding. *Distinction*: A simple trace-based recovery strategy may append raw validator errors to conversation history; LASR-MCP intercepts validator paths for **in-tree JSON Schema recomposition**, splicing missing constraint subtrees directly into the schema on Attempt 2.

Summary of Novelty: LASR-MCP resolves constraints within an already selected tool schema, combining intent-conditioned proactive prediction with validator-derived in-tree recomposition.

VII. Discussion, External Validity, and Limitations

The empirical findings from our primary frozen benchmark (H_1–H_4) and cross-schema study demonstrate that learned adaptive schema resolution with reactive recovery provides an efficient, reliable foundation for enterprise tool agents. Below, we analyze architectural boundaries, validity threats, and operational trade-offs.

A. Architectural Scope and External Validity

1) Schema Topologies and Structural Expressiveness: Enterprise APIs exhibit diverse topologies; our evaluation incorporated nested objects and arrays (e.g., line items in `create_purchase_order`, packages in `shipment_creation`). While JSON Pointer traversal handles nested definitions deterministically, polymorphic branches (`oneOf`, `anyOf`) introduce pruning risks if discriminators are omitted. In LASR, discriminator fields and structural keys are designated invariants (C_I^*), keeping schema branching unambiguous. Automated DAG analysis for recursive schemas remains future work.

2) Schema Validator Standards vs. Business Logic: A core design philosophy of LASR-MCP is strict adherence to JSON Schema Draft 2020-12 specifications [4]. The validator reports error locations as JSON Pointer paths [6] and validation keywords; LASR maps these diagnostics to schema-specific canonical constraint nodes without bespoke heuristics. Enterprise backends enforce transactional business rules (e.g., stock levels, credit checks) outside schema syntax; LASR-MCP explicitly scopes itself to syntactic and structural schema conformance, leaving business failures to application compensation.

3) Model-Family Sensitivities and Parameter Scale: Controlled experiments benchmarked Google's `gemini-3.5-flash-lite` as a representative enterprise model. While pilot ablations observed unconstrained enum generation across evaluated configurations, this empirical observation does not establish that the phenomenon is model-independent, and we make no validated transfer claims to other model families or parameter scales. Conceptually, lean schemas (C_I^*) reduce attention dilution in smaller models while mitigating multi-turn token costs in frontier models; validating cross-family transfer remains an important empirical objective.

4) Unit of Analysis and Constraint Recurrence: Our primary H_1 prediction evaluation ($N = 32$) reflects paired task-level accuracy across held-out template family C. As disclosed in Section IV, all four target constraint combinations recur across train and test splits, and vocabulary overlap is 69.1% due to domain-specific enterprise terminology. A four-cluster sign-flip sensitivity analysis yields $p = 0.125$. Consequently, our findings support lexical predictive accuracy for this held-out template family with recurring constraint bundles, but do not claim inductive generalization to completely novel semantic constraint combinations.

B. Empirical Provenance and Sensitivity Replications

1) Epistemological Separation and Benchmark Provenance: We emphasize strict separation of empirical scopes: **Scope 1 (Primary Frozen Benchmark, $N = 32$ Tasks)**: Evaluates H_1–H_4 on the frozen procurement benchmark (commit `9807340`, SHA-256 `78c2c88e`) under live Google `gemini-3.5-flash-lite` execution, establishing predictive generalization (H_1), invariant safety (H_2), recovery lift (H_3), and regime-dependent token efficiency (H_4). **Scope 2 (Post-Freeze Mechanism Stress Test, $N = 32$ Tasks, 64 Executions)**: Evaluates algorithm and architecture transfer across three new synthetic schemas plus primary replication (four schemas total, SHA-256 `b3d57636`) under a deterministic simulation-based stress test with heuristic token counts, providing preliminary evidence of recovery mechanism robustness without live model API billing or altering the primary benchmark. **Scope 3 (Canonical Live Benchmark, $N = 3,840$ Trials, 7,860 Calls)**: Evaluates full-scale systems performance across 32 tasks, 6 arms, and 20 fixed-seed repetitions on `gemini-3.5-flash-lite` at temperature 0.2, comprising 7,860 response-bearing calls and 154 client transport retries (archive digest `fc82d33b8992914ee130ef2805427ef2a4f01541243e0f93be120e2d20ae7f5b`).

2) Exploratory Live Replication and Sensitivity Analysis: Prior to executing the canonical $N = 3,840$ live benchmark, we conducted an exploratory single-pass replication on the Google Generative Language API using `gemini-3.5-flash-lite` across all 32 tasks and 6 arms to assess endpoint stability. This preliminary run is explicitly distinguished from the definitive $N = 3,840$ dataset by three measurement bounds: (1) recorded executions produced 428 attempt records (285 newly observed unique live response entries written on initial misses alongside approximately 143 within-run prompt-cache reuses); (2) turnaround latencies reflect client logical wall time (including HTTP retries, backoff, and pacing sleeps); and (3) requests did not transmit `seed=42`, and cache records lack provider model-version strings or full `usageMetadata`. Under this exploratory run, observed prompt-plus-candidate token subtotals were: Full Schema 55,549 tokens; CI $k = 0$ 69,555 (-25.21%); CI $k = 1$ static 59,445 (-7.01%); LASR $k = 1$ reactive 52,389 ($+5.69\%$); LASR $k = 2$ reactive 51,224 ($+7.79\%$); and Oracle reactive 53,415 ($+3.84\%$). Conditional mean structural TTS_{struct} tokens among structurally successful tasks were: Full 1,694.7; CI $k = 0$: 2,142.5 (-26.42%); CI $k = 1$ static: 1,738.0 (-2.56%); LASR $k = 1$ reactive: 1,603.7 ($+5.37\%$); LASR $k = 2$ reactive: 1,534.8 ($+9.43\%$); and Oracle: 1,617.0 ($+4.58\%$). First-pass hierarchical success (structural $\wedge$ semantic) was: Full 13/32; CI $k = 0$ 0/32; CI $k = 1$ static 2/32; LASR $k = 1$ reactive 2/32; LASR $k = 2$ reactive 4/32; and Oracle 2/32. Paired bootstrap CIs against Full cross zero in this sample (LASR $k = 1$ $[-13.2\%, +24.4\%]$; LASR $k = 2$ $[-11.0\%, +26.3\%]$), reflecting hosted model variability. We treat this rerun strictly as an exploratory sensitivity replication rather than confusing it with canonical $N = 3,840$ findings.

C. Break-Even Boundary and Analytical Scaling Projections

1) The Retry-Amortization Break-Even Boundary: The full-scale $N = 3,840$ live benchmark (32 tasks, 6 arms, 20 fixed-seed executions per task-arm pair at temperature 0.2) characterizes an observed *Retry-Amortization Boundary* across the four evaluated enterprise task archetypes in agentic tool-calling. Let P_{base} denote base user prompt tokens, S_{full} monolithic schema tokens, S_0 upfront lean schema tokens, and ΔS reactive constraint expansion. Let $C_{\text{gen},1}$ and $C_{\text{gen},2}$ denote candidate generation lengths across turns (with $C_{\text{gen},1} \approx C_{\text{gen},2} \approx C_{\text{gen}}$). In single-turn success ($T = 1$), token savings is strictly positive ($\Delta\text{Tok} = S_{\text{full}} - S_0 > 0$). When an omitted constraint forces a validation error and retry ($T = 2$), the request re-transmits base prompt P_{base}, expanded schema $S_0 + \Delta S$, prior history H_1 ($|H_1| \approx C_{\text{gen}}$), and validator feedback F_1, yielding modeled two-turn cost:

$$\text{Cost}(T = 2) \approx 2P_{\text{base}} + 2S_0 + \Delta S + H_1 + F_1 + 2C_{\text{gen}} \quad (14)$$

which exceeds the single-turn monolithic cost $\text{Cost}_{\text{full}}(T = 1) = P_{\text{base}} + S_{\text{full}} + C_{\text{gen}}$ whenever:

$$P_{\text{base}} + S_0 + \Delta S + H_1 + F_1 + C_{\text{gen}} > S_{\text{full}} - S_0 \quad (15)$$

Terms in this derivation represent modeled analytical estimates. When retry turn overhead ($P_{\text{base}} + S_0 + \Delta S + H_1 + F_1 + C_{\text{gen}}$) exceeds the upfront compression delta ($S_{\text{full}} - S_0$) as formalized in Inequality (15), a multi-turn retry cascade eliminates the initial token savings. Note that a simpler reactive baseline that falls back to the full monolithic schema ($S_{\text{full}} = 552.0$ tok) on Attempt 2 rather than splicing only the violated delta ($S_0 + \Delta S = 404.6$ tok) would incur an additional $+147.4$ schema tokens per retry turn.

Empirically across our 32 task clusters, this boundary creates a sharp domain divergence: (1) **Routine Regimes** ($C_1, m_{\textbf{task}}(x) \leq k$): In Cluster 1 (`routine_regional_replenish`), active task constraints fall within the $k = 2$ upfront bundle (75.0% first-pass success). Initial schema compression (359.4 vs. 552.0 tokens) is preserved without round-trip penalties, delivering approximately **13.5**% lower $P + C$ token usage (847.6 vs. 979.8 mean recorded tokens) at 100.0% structural reliability (signed savings: $+13.5\%$). We do not generalize this cluster-specific result to the full benchmark. (2) **Deep Inter-Field Regimes** (C_2–$C_4, m_{\textbf{task}}(x) > k$): In complex procurement tasks with cross-field dependencies, omitted constraints trigger Attempt 1 failures (0.0% first-pass). Each retry re-transmits prompt and prior generation, incurring multi-turn penalties that inflate net workload tokens by $+22.2\%$ (C_2, $1,557.5$ vs. $1,274.2$; signed savings: -22.2%), $+59.8\%$ (C_3, $1,610.9$ vs. $1,008.1$; signed savings: -59.8%), and $+33.6\%$ (C_4, $1,609.9$ vs. $1,205.0$; signed savings: -33.6%) relative to Full Control. Across the full 32-task benchmark ($N = 3,840$ trials), Full Control achieves a mean recorded subtotal of $1,116.8$ tokens versus $1,406.5$ for LASR $k = 2$ (workload increase: $+25.94\%$, 95% CI: $[+3.16\%, +52.55\%]$; signed relative savings: -25.94%, 95% CI: $[-52.55\%, -3.16\%]$, bootstrap CI excluding zero); we do not describe the overall benchmark result as token savings.

2) Multi-Tool Catalog Analytical Projections: We emphasize that our empirical evaluation evaluated single-tool isolation with deep constraint hierarchies. In multi-tool agent registries where an agent is exposed to $W > 1$ heterogeneous enterprise tools $\mathcal{T} = \{t_1, \ldots, t_W\}$ concurrently (e.g., $W = 10$ or $W = 50$ enterprise tools in an MCP catalog), each tool i possesses monolithic schema size $S_{\text{full},i}$ and pruned lean schema size $S_{0,i}$. The gross upfront catalog savings from compressing all W tools simultaneously and net savings under single-tool invocation failure are modeled as:

$$\text{Savings}_{\text{catalog}}^{\text{gross}} = \sum\nolimits_{i=1}^{W} \big(S_{\text{full},i} - S_{0,i}\big), \quad (16)$$

$$\text{Savings}_{\text{net}} = \text{Savings}_{\text{catalog}}^{\text{gross}} - \mathbb{P}(\text{retry}) \cdot \text{Penalty}_{\text{retry}}. \quad (17)$$

Under homogeneous average tool dimensions ($S_{\text{full},i} \approx \bar{S}_{\text{full}}$, $S_{0,i} \approx \bar{S}_0$), this scales as $W \cdot (\bar{S}_{\text{full}} - \bar{S}_0)$. Net savings remains strictly positive whenever $\sum_{i=1}^{W}(S_{\text{full},i} - S_{0,i}) > \mathbb{P}(\text{retry}) \cdot \text{Penalty}_{\text{retry}}$, where $\text{Penalty}_{\text{retry}} \approx P_{\text{base}} + S_0 + \Delta S + H_1 + F_1 + C_{\text{gen}}$. Because gross upfront savings scales linearly with W while the retry penalty applies only to the single invoked tool, multi-tool registries are analytically projected to yield substantial net savings even under modest retry rates. We restrict this scaling relationship strictly to an analytical projection and future research direction, while keeping our formal empirical claims strictly grounded in the single-tool isolation dataset ($N = 3,840$).

D. Limitations & Methodological Disclosures

We explicitly disclose four methodological boundaries of this study: (1) **Fixed-Seed Determinism**: All 20 repetitions per task-arm combination produced identical first responses and outcomes under the repeated fixed seed (temperature 0.2, seed 42), verifying execution stability rather than representing independent stochastic draws. (2) **Data Collection Timeline**: As detailed in Section III-D, collection spanned multiple days and sessions across paid and quota-limited API accounts (completing 3,149 initial trials followed by a 2.8-hour final session completing the remaining 691 trials across backup keys). (3) **Single-Tool Scope**: Empirical metrics reflect single-tool isolation rather than concurrent multi-tool exposure. (4) **Model Specificity**: Live evaluations reflect Google's `gemini-3.5-flash-lite`; cross-family transfer remains subject to future work.

VIII. Conclusion

This study resolves an architectural question in agentic tool calling: when does lazy schema compression save tokens, and when do reactive retry cascades outweigh upfront reduction? Across $3,840$ completed empirical trials (32 tasks, 6 arms, 20 fixed-seed executions at temperature 0.2), naive constraint stripping (`ci_k0`) proved an operational anti-pattern that inflates net token consumption by 84.3% while leaving 12.5% of workflows structurally broken. By introducing prioritized $k = 2$ upfront constraint bundling coupled with monotonic reactive subtree slicing (`ci_k2_reactive`), LASR-MCP cuts token waste by 31.7% over naive lean baselines, restores 96.9% structural completion, and operates within 2.1% of a task-aware

oracle baseline (2.08% higher relative to oracle). Crucially, our break-even analysis identifies an operational boundary governed by the ratio of task-conditioned constraints $m_{\text{task}}(x)$ to upfront budget k: in routine regimes ($C_1, m_{\text{task}}(x) \leq k$), LASR $k = 2$ delivers approximately 13.5% lower $P+C$ token usage (847.6 vs. 979.8 mean tokens) at 100.0% reliability. Conversely, across the full 32-task benchmark, Full Control achieves a mean recorded subtotal of $1,116.8$ tokens versus $1,406.5$ for LASR $k = 2$ (workload increase: $+25.94\%$, 95% CI: $[+3.16\%, +52.55\%]$; signed relative savings: -25.94%, 95% CI: $[-52.55\%, -3.16\%]$, bootstrap CI excluding zero). Monolithic schemas thus remain optimal for isolated single-tool calls with deep cross-field invariants (C_2–C_4, $m_{\text{task}}(x) > k$) under single-tool objectives by avoiding retry cascades unless amortized across multi-tool registries. These empirical boundaries equip enterprise MCP architects with concrete, mathematically grounded deployment rules.

Declaration of Generative AI and AI-Assisted Technologies

In accordance with IEEE and academic publishing guidelines on Generative AI transparency, the author discloses: (1) As the primary empirical subject, live API calls were executed against Google's `gemini-3.5-flash-lite` model ($N = 3,840$ trials, $7,860$ API invocations) to evaluate tool-calling schema resolution under the `mcp-inventory-bench` v2.0.3 protocol. (2) During manuscript preparation, AI-assisted coding and editing tools were utilized strictly for LaTeX typesetting, copy-editing for readability, and generating deterministic plotting scripts (`TikZ`/`PGFPlots` and `matplotlib`) directly from verified CSV/JSON telemetry. No generative AI image-synthesis tools were used to create or alter figures. The author reviewed, audited, and cryptographically verified all code, empirical data, statistical bootstrap claims, and visual representations, assuming full responsibility for manuscript integrity.

References

[1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," *International Conference on Learning Representations (ICLR)*, 2023.

[2] T. Schick, J. Dwivedi-Yu, S. Patil, G. Wenzek, P. Ludwig, M. Lewis, L. Zettlemoyer, and S. Riedel, "Toolformer: Language models can teach themselves to use tools," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.

[3] Model Context Protocol Working Group, "Model Context Protocol specification (version 2026-07-28)," jul 2026. [Online]. Available: https://modelcontextprotocol.io/specification

[4] A. Wright, H. Andrews, B. Hutton, and G. Dennis, "JSON schema: A media type for describing JSON documents (draft 2020-12)," jun 2022. [Online]. Available: https://json-schema.org/draft/2020-12/json-schema-core.html

[5] S. G. Patil, T. Zhang, X. Wang, and J. E. Gonzalez, "Gorilla: Large language model connected with massive APIs," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 37, 2024, pp. 126 544–126 565.

[6] P. Bryan, K. Zyp, and M. Nottingham, "Javascript object notation (JSON) pointer," RFC 6901, 2013.

[7] T. Gan and Q. Sun, "Mitigating prompt bloat in LLM tool selection via retrieval-augmented generation," *arXiv preprint arXiv:2505.03275*, 2025.

[8] X. Fei, X. Zheng, and H. Feng, "MCP-Zero: Active tool discovery for autonomous LLM agents," *arXiv preprint arXiv:2506.01056*, 2025.

[9] Y. Qin, S. Liang, Y. Ye, K. Zhu, L. Yan, Y. Lu, Y. Lin, X. Cong, X. Tang, B. Qian *et al.*, "ToolLLM: Facilitating large language models to master 16000+ real-world APIs," *arXiv preprint arXiv:2307.16789*, 2023.

[10] A. Sadani and D. Kumar, "Tool attention is all you need: Dynamic tool gating and lazy schema loading for eliminating the MCP/tools tax in scalable agentic workflows," *arXiv preprint arXiv:2604.21816*, 2026.

[11] F. Sakizli, "TSCG: Deterministic tool-schema compilation for agentic LLM deployments," *arXiv preprint arXiv:2605.04107*, 2026.

[12] H. Jiang, Q. Wu, C.-Y. Lin, Y. Yang, and L. Qiu, "LLMLingua: Compressing prompts for accelerated inference of large language models," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023, pp. 13 358–13 376.

[13] Z. Pan, Q. Wu, H. Jiang, M. Meng, Y. Yang, and L. Qiu, "LLMLingua-2: Data distillation for efficient and faithful task-agnostic prompt compression," in *Findings of the Association for Computational Linguistics: ACL 2024*, 2024, pp. 8252–8269.

[14] Y.-e. Cho, "SchemaRouter: Field-aware tool routing for efficient heterogeneous agentic RAG," *arXiv preprint arXiv:2608.21375*, 2026.

[15] Z. Zhai, X. Tan, G. Zou, X. Wang, and W. Zhang, "HyperAgent: Planning and acting over tool-schema hypergraphs for tool-use LLM agents," *arXiv preprint arXiv:2608.02650*, 2026.

[16] H. Xia, Y. Li, C. Du, M. Song, and W. Li, "ToolSpec: Accelerating tool calling via schema-aware and retrieval-augmented speculative decoding," in *Proceedings of the 2026 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2026, pp. 9401–9414.